

VestiFlow — Guida operatore, proprietario e sviluppatore

Versione documento: 1.6 — Giugno 2026

Destinatari: operatori piattaforma VestiFlow (`isPlatformAdmin`), proprietario del prodotto, sviluppatori che mantengono il gestionale.

Visibilità: accessibile solo come **Guida tecnica** (`/app/admin/guide`) con account il cui email è in `PLATFORM_ADMIN_EMAILS` . I clienti negozio usano la voce **Guida** (`/app/guide`) con il manuale operativo del negozio.

Indice operatore

1. [Ruolo operatore piattaforma](#)
 2. [Architettura e stack](#)
 3. [Modello multi-tenant](#)
 4. [Struttura repository](#)
 5. [Ambiente di sviluppo locale](#)
 6. [Variabili d'ambiente](#)
 7. [Onboarding nuovo cliente \(tenant\)](#)
 8. [Autenticazione e ruoli](#)
 9. [Integrazione Shopify \(tecnica\)](#)
 10. [Supabase: database, Auth, Storage, RLS](#)
 11. [Dominio dati principale](#)
 12. [API e permessi tenant](#)
 13. [Import ed export CSV](#)
 14. [Scanner barcode](#)
 15. [Generazione guide \(HTML/PDF\)](#)
 16. [Build, test e CI](#)
 17. [Deploy produzione](#)
 18. [Sicurezza e checklist pre-release](#)
 19. [Troubleshooting tecnico](#)
 20. [Limitazioni note e roadmap](#)
-

1. Ruolo operatore piattaforma

L'**operatore piattaforma** (tu, come proprietario/sviluppatore) non coincide con il **Titolare** di un tenant negozio.

Concetto	Dove vive	Come si ottiene
Platform admin	Flag <code>isPlatformAdmin</code> nel profilo utente API	Email in <code>PLATFORM_ADMIN_EMAILS</code> (backend)
Titolare tenant	Ruolo <code>owner</code> nel DB tenant	Scelto al provisioning in Nuovo cliente
Ruoli negozio	<code>owner</code> , <code>admin</code> , <code>manager</code> , <code>clerk</code>	Assegnati al create tenant; non modificabili self-service oggi

Cosa vede un platform admin in UI

Shell **dedicata** (non le schermate operative del negozio):

Voce menu	Route	Contenuto
Clienti	<code>/app/admin/clients</code>	Elenco tenant + form Nuovo cliente
Impostazioni	<code>/app/admin/account</code>	Profilo operatore, foto, MFA, tema
Guida tecnica	<code>/app/admin/guide</code>	Questo manuale

In topbar: **tema**, **avatar** (clic → Impostazioni), **Esci**. **Nessun** selettore sede né indicatore sync Shopify (non sei dentro un tenant negozio).

Per gestire catalogo/magazzino di un cliente: accedi con le **credenziali del titolare** di quel tenant (account negozio), non con l'account operatore piattaforma.

Backend

Endpoint sotto `/api/v1/admin/tenants` protetti da `JwtAuthGuard` + `PlatformAdminGuard` (verifica email contro env).

2. Architettura e stack

Componente	Tecnologia	Ruolo
Frontend	Angular 21+ (standalone, signals, OnPush)	SPA/PWA gestionale
Hosting frontend	Firebase App Hosting (o equivalente static)	CDN + HTTPS
Backend API	NestJS	Business logic, Shopify OAuth, webhook
Hosting API	Railway (tipico)	Node 22, env secrets
Database	PostgreSQL via Supabase	Dati multi-tenant
Auth	Supabase Auth (JWT HS256)	Login email/password, MFA opzionale
Storage immagini	Supabase Storage bucket <code>product-media</code>	Upload media prodotti
Storage avatar	Supabase Storage bucket <code>user-avatars</code>	Foto profilo utente (public)
E-commerce	Shopify Admin API + webhook	Catalogo, stock, ordini, clienti
E-commerce alt.	TikTok Shop Open API + OAuth (parziale)	Catalogo push + giacenze — early stage

Regola: il frontend **non** contiene secret Shopify né service role Supabase. Tutto passa dall'API.

3. Modello multi-tenant

- Ogni **tenant** = un'azienda cliente = **un canale e-commerce** collegato (Shopify, TikTok Shop) oppure **solo gestionale**.
- Campo `channelProfile` sul tenant: `gestionale` | `shopify` | `tiktok_shop` — determina quali pannelli Integrazione compaiono in Impostazioni lato cliente.
- Isolamento dati: colonna `tenantId` su entità business; filtro obbligatorio lato API.
- Location** ≠ **Store**: lo stock è per location (semantica Shopify); lo store è entità commerciale.
- Variante** = unità minima inventario (SKU univoco interno).
- Due shop Shopify distinti** → **due tenant** VestiFlow separati.
- Più location nello stesso shop** → un tenant, N location sincronizzate.

4. Struttura repository

```
vestiflow/
├── src/app/           # Frontend Angular
│   ├── core/         # Auth, guards, permissions, HTTP
│   ├── shared/        # Componenti UI riutilizzabili
│   ├── features/      # Feature lazy-loaded (products, inventory, sales, admin, guide...)
│   └── layout/        # Shell sidebar + topbar
├── api/              # Backend NestJS
│   ├── src/admin/     # Provisioning tenant (platform admin)
│   ├── src/products/  # Catalogo, CSV import/export
│   ├── src/inventory/ # Giacenze, movimenti, CSV
│   ├── src/shopify/   # OAuth, sync, webhook, rate limit
│   └── prisma/        # Schema e migration PostgreSQL
├── docs/             # Guide Markdown + HTML/PDF
├── public/guide/      # Guida utente in-app (pubblica)
├── src/assets/guide-admin/ # Guida tecnica (solo route admin)
└── scripts/          # generate-guide-*, check-rls, environment prod
```

Alias TypeScript: `@core/*`, `@shared/*`, `@features/*`, `@env/*`.

5. Ambiente di sviluppo locale

Requisiti

- Node.js **22** LTS (`.nvmrc`)
- npm (lockfile committato)
- Progetto Supabase (URL + anon key + service role per API)
- Account Shopify Partners (app custom) per test OAuth

Avvio

```
# Frontend (porta 4200)
npm install
npm start

# Backend (porta 3000)
cd api
cp .env.example .env # compilare variabili
npm install
npx prisma migrate dev
npm run start:dev
```

Shopify in locale

Shopify richiede URL **pubblici** per OAuth callback e webhook. Usa **ngrok** o **cloudflared**:

1. Tunnel verso `localhost:3000`
2. Imposta `SHOPIFY_APP_URL` e `FRONTEND_URL` nel `api/.env`
3. Aggiorna redirect URL nell'app Partners

PWA locale

```
npm run build:pwa:local
npm run serve:pwa
```

6. Variabili d'ambiente

Frontend (`.env` → **build**, valori **pubblici**)

Vedi `.env.example` in root:

Variabile	Significato
<code>VESTIFLOW_API_BASE_URL</code>	URL API produzione
<code>VESTIFLOW_SUPABASE_URL</code>	URL Supabase
<code>VESTIFLOW_SUPABASE_ANON_KEY</code>	Anon/publishable key
<code>VESTIFLOW_ENABLE_BARCODE_SCANNER</code>	Feature flag scanner
<code>VESTIFLOW_ENABLE_SHOPIFY</code>	Feature flag integrazione Shopify UI

Mai service role o secret Shopify nel frontend.

Backend (`api/.env`)

Vedi `api/.env.example` :

Variabile	Significato
DATABASE_URL / DIRECT_URL	PostgreSQL Supabase
SUPABASE_URL , SUPABASE_SERVICE_ROLE_KEY , SUPABASE_JWT_SECRET	Auth e admin DB
CORS_ORIGINS	Origini frontend consentite
SHOPIFY_*	OAuth, scope, cifratura token, rate limit
PLATFORM_ADMIN_EMAILS	Email operatori (virgola)
SUPABASE_PRODUCT_MEDIA_BUCKET	Bucket immagini prodotto (product-media)
SUPABASE_USER_AVATARS_BUCKET	Bucket foto profilo (user-avatars)
TIKTOK_*	OAuth TikTok Shop, cifratura token, API base
FRONTEND_URL	Redirect post-OAuth

7. Onboarding nuovo cliente (tenant)

UI: `/app/admin/clients` (solo platform admin) — tabella **Clienti registrati** e pulsante **Nuovo cliente**.

Procedura creazione

1. **Identificazione** — nome commerciale, ragione sociale opzionale
2. **Anagrafica** — P.IVA, CF, sede, contatti (opzionali)
3. **Profilo canale** — **Solo gestionale**, **Shopify** o **TikTok Shop** (determina integrazioni visibili al cliente)
4. **Primo accesso** — ruolo VestiFlow (`owner` default), nome, email, **password iniziale** (scelta dall'admin)
5. **Setup** — nome negozio e location iniziale (opzionali; default «Negozio principale»)

Il backend crea: tenant, utente Supabase Auth con password, store, location, profilo utente collegato.

Dopo il provisioning

Consegna credenziali al titolare **in modo sicuro** (email + password). Il titolare accede da `/login` e può cambiare password da Impostazioni o tramite «Password dimenticata».

Invito email (standby): il flusso con invito Supabase al titolare esiste nel codice ma è **disabilitato** finché non è attivo Supabase a pagamento con SMTP. Riattivazione: variabile Railway SUPABASE_OWNER_EMAIL_INVITE=true + configurazione redirect/template Supabase (vedi commit/feature invito).

Il titolare completa in base al profilo canale:

Profilo canale	Passi titolare
Shopify	MFA → OAuth Shopify → sync location → webhook → import
TikTok Shop	MFA → OAuth TikTok Shop → verifica location
Solo gestionale	MFA → catalogo e magazzino solo in VestiFlow

Modifica tenant esistente

Tabella **Clienti registrati** → click riga → `/app/admin/clients/:id`.

Modificabile: anagrafica, **profilo canale** (se nessuna integrazione attiva), nome titolare, negozio, location. **Email** e **ruolo** del primo utente sono **sola lettura** in UI (campi disabilitati).

Per cambiare profilo canale con integrazione già connessa: il cliente deve **disconnettere** Shopify o TikTok da Impostazioni prima.

Eliminazione tenant (zona pericolosa)

In **Modifica cliente**, pannello **Zona pericolosa** → **Elimina cliente**: rimuove tenant, dati negozio, utenti e integrazioni. Operazione **irreversibile** con dialog di conferma.

API

Metodo	Path	Azione
GET	<code>/admin/tenants</code>	Lista tenant
POST	<code>/admin/tenants</code>	Crea tenant + primo utente
GET	<code>/admin/tenants/:id</code>	Dettaglio
PATCH	<code>/admin/tenants/:id</code>	Aggiorna anagrafica/setup
DELETE	<code>/admin/tenants/:id</code>	Elimina tenant e dati

Body create include `role` (`owner` | `admin` | `manager` | `clerk`) e `channelProfile` (`gestionale` | `shopify` | `tiktok_shop`).

8. Autenticazione e ruoli

Flusso auth

1. Login Supabase Auth (frontend)
2. JWT inviato all'API (`Authorization: Bearer`)
3. `JwtAuthGuard` valida firma con `SUPABASE_JWT_SECRET`
4. Profilo utente + `tenantId` + ruolo + `isPlatformAdmin` caricati

Ruoli tenant (UI + API)

Implementati in `tenant-permissions.util.ts` (frontend) e guard analoghi lato API.

Permesso chiave	Ruoli
Shopify sync / import catalogo	owner, admin
CRUD prodotti, CSV catalogo	owner, admin, manager
CSV giacenze, ordini fornitori	owner, admin, manager
Movimenti magazzino, inventario fisico	tutti
MFA settings	owner, admin, manager

Route Angular sensibili: `tenantRoleGuard` + `data.tenantRoutePermission`.

Foto profilo utente

Metodo	Path	Azione
POST	<code>/auth/avatar</code>	Upload foto (multipart <code>file</code>)
DELETE	<code>/auth/avatar</code>	Rimuove foto e file su Storage

Validazione: JPEG/PNG/WebP, max 2 MB. Bucket `user-avatars` (public). Dopo upload/delete invalida cache profilo JWT (`AuthProfileCacheService.invalidate`).

UI: **Impostazioni** → **Profilo** (tenant) e **Impostazioni** operatore (`/app/admin/account`). Avatar in topbar cliccabile → Impostazioni.

9. Integrazione Shopify (tecnica)

Ownership sync (riepilogo)

Entità	Owner	Note
Catalogo prodotti VestiFlow (catalogOrigin=vestiflow)	VestiFlow	CRUD completo; push al save; delete write-through verso Shopify se shopifyProductId
Catalogo prodotti Shopify (catalogOrigin=shopify)	Shopify	Pull import/webhook; in VF solo PATCH stagione + purchasePriceMinor ; no delete/sync manuale/media
Clienti, ordini online	Shopify	Read-only in VF
Giacenze	Condiviso	VF: carichi/rettifiche; Shopify: vendite
Location	Shopify master	Import + mapping; cleanup sedi stale
Ordini fornitori	Solo VestiFlow	—
Anagrafica tenant	Solo VestiFlow (admin)	GET /tenant/company read-only in UI Sede fisica

Origine catalogo (catalogOrigin / shopifyCatalogLinkKind)

Campi su Product (migration 0016 + 0017):

Valore	Significato	Impostato quando
catalogOrigin=vestiflow	Il gestionale possiede i campi catalogo condivisi	Create in VF, import CSV, push verso Shopify (linkKind=pushed)
catalogOrigin=shopify	Shopify Admin possiede il catalogo	Import pull (linkKind=imported) o backfill legacy
shopifyCatalogLinkKind=imported	Collegato da import Shopify → VF	ShopifyProductPullService
shopifyCatalogLinkKind=pushed	Creato in VF e inviato al canale	Create/push ShopifyProductPushService , import CSV

Logica centralizzata in `api/src/products/catalog-origin.util.ts`:

- `isVestiflowCatalogOwner()` — esclude import Shopify e legacy con link alla creazione; include push e prodotti con media locale Supabase
- `shouldSkipShopifyCatalogImport()` — pull/webhook **non sovrascrivono** catalogo VF-owned
- `assertShopifyCatalogUpdateAllowed()` — su `shopify`: blocca mutazioni catalogo; consente `season` + `purchasePriceMinor`
- `assertShopifyCatalogDeleteAllowed()` / `assertShopifyCatalogManualSyncAllowed()` / `assertShopifyCatalogMediaMutationAllowed()`

Backfill tenant esistenti (dopo deploy migration):

```
cd api && npm run backfill:catalog-origin      # dry-run
cd api && npm run backfill:catalog-origin:apply # scrivo su DB
```

UI tenant: colonna **Fonte** in lista prodotti; badge `Fonte: VestiFlow` / `Fonte: Shopify` in dettaglio; form in modalità **Modifica dati operativi** se `catalogOrigin=shopify` (`catalog-origin.util.ts` FE + `product-form` / `product-detail`).

Taxonomy Shopify e metafield di categoria

Picker e attributi categoria nel form prodotto (step **Dati generali**).

Layer	Percorso / componente
API	<code>GET /shopify/taxonomy/categories</code> , <code>GET /shopify/taxonomy/category-attributes?categoryId=</code>
BE	<code>ShopifyTaxonomyService</code> , <code>ShopifyCategoryMetafieldsService</code> , util <code>shopify-category-metafields.util.ts</code>
FE picker categoria	<code>shopify-taxonomy-picker</code>
FE attributi	<code>shopify-category-attributes</code> (in <code>product-general-step</code>)
Persistenza	<code>Product.shopifyCategoryMetafields</code> (JSON), <code>shopifyTaxonomyCategoryId</code> / <code>FullName</code>

Modello `shopifyCategoryMetafields`: array di oggetti con campi `attributeId`, `attributeName`, `namespace`, `key`, `metafieldType` e array `values`. Ogni attributo può avere **più valori** in `values` (allineato a Shopify).

UI multi-valore: se `metafieldType` inizia con `list`. → `SelectMenuComponent` in modalità `multiple` (`isShopifyCategoryMetafieldMultiValue` in `shopify-category-metafield.util.ts`). Altrimenti select singola.

Push: `ShopifyProductPushService` → `pushCategoryMetafields` serializza i GID taxonomy (`serializeTaxonomyValueListGids`) o crea metaobject per tipo `list.metaobject_reference`. Import/pull popola lo stesso JSON da metafield prodotto Shopify.

Cambio negozio e purge dati Shopify

Modulo `ShopifyShopChangeService` + wizard FE `shopify-shop-change-wizard`.

Endpoint	Metodo	Ruolo	Scopo
<code>/shopify/shop-change/preview</code>	GET	owner/admin	Conteggi dati collegati a Shopify + blockers (es. ordini fornitori aperti su location Shopify)
<code>/shopify/shop-change/purge</code>	POST	owner/admin	Rimuove dati importati/syncati da Shopify (prodotti, varianti, clienti, ordini vendita, location collegate, giacenze/movimenti associati)
<code>/shopify/connection</code>	DELETE	owner/admin	Disconnessione OAuth (token revocato); non purge catalogo

Flussi UI:

- **Cambia negozio** — wizard mode `shop-change`: preview → opzione purge → conferma dominio → disconnect → redirect OAuth nuovo shop
- **Disconnetti e rimuovi dati** — wizard mode `disconnect`: preview → purge opzionale → disconnect
- **Disconnetti Shopify** — solo DELETE connection, dati locali restano

OAuth guard: tentativo di collegare un dominio diverso da quello attivo senza purge precedente → errore esplicito (evita fork silenzioso tra shop).

Cosa NON cancella il purge: ordini fornitori (salvo blocker se legati a location Shopify), anagrafica tenant (**Sede fisica**), utenti tenant, movimenti non legati a entità Shopify rimosse.

Eliminazione prodotto

`ProductsService.delete()`:

1. `assertShopifyCatalogDeleteAllowed()` — se `catalogOrigin=shopify` → **409** con messaggio utente (elimina da Shopify Admin)
2. Verifica assenza movimenti stock sul prodotto
3. Se `shopifyProductId` presente e `catalogOrigin=vestiflow` → `ShopifyProductPushService.deleteProduct()` **prima** del delete DB
4. Errori mappati: `not_connected`, `missing_write_products_scope`, `shopify_error` → **422** con messaggio utente; DB intatto

Scope richiesto per delete write-through: `write_products` (incluso in `SHOPIFY_SCOPES` default).

FE: pulsante **Elimina** e **Sincronizza con Shopify** nascosti se `catalogOrigin=shopify`.

Sync location — comportamento post-fix

`ShopifyLocationSyncService`:

- Importa/aggiorna location da Shopify; **non** collega più automaticamente la sede onboarding `LOC-01` al primo match Shopify
- `buildLinkedLocationData` aggiorna il **nome** da Shopify a ogni re-sync
- `cleanupStaleShopifyLocations` rimuove location non più presenti su Shopify API
- `removeEmptyOnboardingLocation` elimina sede temporanea onboarding vuota e non collegata dopo sync
- Dopo sync/disconnect/purge: `ShopifyConnectionRefreshService.notifyInvalidated()` → shell ricarica location e topbar

FE: `filterLocationsForTopbar` nasconde sede onboarding locale quando Shopify è connesso.

Anagrafica tenant (Sede fisica)

Endpoint	Metodo	Scopo
<code>/tenant/company</code>	GET	Dati commerciali tenant (name, storeName, PIVA, indirizzo, contatti) — read-only lato cliente

Popolati al provisioning admin (`create-client`). UI: `tenant-client-card` in Impostazioni.

OAuth

- Scope default in `SHOPIFY_SCOPES` (`.env.example`)
- Token cifrati at rest (`SHOPIFY_TOKEN_ENCRYPTION_KEY`)
- Diagnostica scope in Impostazioni: richiesti vs concessi

Webhook

Registrati con **Attiva aggiornamenti automatici**. Idempotenza lato backend per eventi duplicati/out-of-order.

Limiti Shopify Admin API — concetto

Shopify **non addebita le chiamate API** a consumo. Il piano del negozio definisce **quanto velocemente** puoi chiamare le Admin API prima di essere **rallentato** (HTTP 429 **Too Many Requests** / throttling). Non esiste un contatore "a pagamento per chiamata".

VestiFlow usa oggi soprattutto la **REST Admin API** (prodotti, location, inventario, immagini).

Documentazione ufficiale Shopify:

- [Limiti API \(panoramica\)](https://shopify.dev/docs/api/usage/limits) (https://shopify.dev/docs/api/usage/limits)
- [Rate limit REST Admin API](https://shopify.dev/docs/api/admin-rest/usage/rate-limits) (https://shopify.dev/docs/api/admin-rest/usage/rate-limits)

Fasce tipiche REST (piano negozio → velocità massima):

Piano Shopify (indicativo)	REST Admin API
Basic / Grow (Standard)	~ 2 richieste/sec , bucket 40
Advanced	~ 4 richieste/sec , bucket 80
Plus	limiti più alti (fascia Enterprise)

Il header `X-Shopify-Shop-API-Call-Limit` (es. `32/40`) indica quanto del bucket è consumato.

Cosa fa VestiFlow quando si avvicina o supera il limite

Implementazione in `api/src/shopify/`:

Componente	Ruolo
<code>ShopifyRateLimiterService</code>	Throttle per shop prima di ogni richiesta outbound
<code>ShopifyAdminClient.request()</code>	Applica throttle, legge header bucket, retry su 429
<code>ShopifyProductPullService</code>	Mutex import catalogo (un import per tenant alla volta, process-local)

Comportamento:

1. **Intervallo minimo** tra richieste (default 550 ms $\approx$ 1,8 req/s, sotto il limite Basic 2/s).
2. Se `X-Shopify-Shop-API-Call-Limit` $\geq$ soglia (**85%** del bucket), pausa **1 s** prima delle richieste successive.
3. Su **HTTP 429**: legge `Retry-After`, backoff esponenziale, fino a **5** retry; poi errore 429 con messaggio utente: «*Shopify ha limitato temporaneamente le richieste API...*».
4. **Import catalogo**: enrichment leggero (`skipRemoteMetadata: true`, niente costi varianti extra) per ridurre chiamate; **non avviare due import** sullo stesso tenant in parallelo.
5. **Webhook** prodotti/ordini/giacenze: **0 chiamate outbound** — Shopify invia i dati a VestiFlow.

Variabili env (REST outbound):

Variabile	Default	Effetto
<code>SHOPIFY_API_MIN_INTERVAL_MS</code>	550	Pausa minima tra richieste
<code>SHOPIFY_API_MAX_RETRIES</code>	5	Retry su HTTP 429
<code>SHOPIFY_API_BUCKET_HIGH_WATERMARK</code>	0.85	Pausa se secchio pieno
<code>SHOPIFY_API_BUCKET_PAUSE_MS</code>	1000	Durata pausa secchio

Nota deploy: il rate limiter Shopify è **process-local**. Con più repliche Railway ogni istanza ha il proprio contatore (comportamento conservativo, non centralizzato).

Chiamate indicative per operazione:

Operazione	Chiamate Shopify (ordine di grandezza)	Note
Import catalogo (Shopify → VF)	~4 per 1000 prodotti (pagine da 250)	Lento con cataloghi grandi: normale ; imposta <code>catalogOrigin=shopify</code>
Webhook prodotti/giacenze/ordini	0 outbound	Event-driven; skip catalogo se <code>vestiflow-owned</code>
Push prodotto al save (VF → Shopify)	1–N per prodotto	Solo <code>catalogOrigin=vestiflow</code> ; imposta <code>linkKind=pushed</code>
Delete prodotto (VF → Shopify)	1	Solo <code>catalogOrigin=vestiflow</code> + <code>shopifyProductId</code>
Purge dati Shopify	0 outbound (delete DB locale)	Dopo purge, riconnessione OAuth
Push giacenza dopo movimento	~1 per movimento	Write-through inventario
Sync location	1	Trascurabile

Cosa fare se un cliente vede errori 429:

1. Non ripremere import/sync in loop — attendere 1–2 minuti e **un solo** retry.
2. Evitare import catalogo + sync massivo contemporanei sullo stesso negozio.
3. In Railway, verificare env rate limit (non abbassare `SHOPIFY_API_MIN_INTERVAL_MS` sotto 500 ms su piani Basic).
4. Cataloghi molto grandi: l'import può richiedere diversi minuti; il backend riprova da solo fino al limite retry.

Non ancora implementato (roadmap): coda lavori persistente in background per bulk multi-tenant; oggi le operazioni massicce sono serializzate per shop/tenant ma restano sincrone nella request HTTP.

Rate limiting (REST Admin API) — riferimento rapido env

Vedi tabella variabili sopra. Valori in `api/.env.example`.

Import catalogo: enrichment ridotto (`skipRemoteMetadata`) + mutex un import per shop.

Troubleshooting `read_products`

1. App Partners: versione attiva include `read_products` , `read_inventory`
2. `SHOPIFY_API_KEY` Railway = Client ID app
3. Disinstalla app dal negozio Shopify
4. Disconnetti + riconnetti in VestiFlow
5. Verifica riga **Catalogo prodotti** → **Lettura** in Impostazioni

Protected customer data

Ordini/clienti webhook possono richiedere approvazione Partners. Catalogo/giacenze non dipendono da questo.

Integrazione TikTok Shop (tecnica)

Stato: early / parziale. OAuth + push catalogo (create/update) + push giacenze dopo movimenti VF. Non implementati: import da TikTok, webhook, vendite/clienti, parità funzionale con Shopify. Aggiornare questa sezione quando l'integrazione sarà completa.

Modulo `api/src/tiktok/` (OAuth, connessione, sync catalogo e inventory push).

Aspetto	Dettaglio
Profilo tenant	Solo tenant <code>channelProfile = tiktok_shop</code> vedono pannello Impostazioni
OAuth	Partner Center → env <code>TIKTOK_APP_KEY</code> , <code>TIKTOK_APP_SECRET</code> , <code>TIKTOK_SERVICE_ID</code>
Token	Cifrati at rest (<code>TIKTOK_TOKEN_ENCRYPTION_KEY</code>)
Sync scope	Push prodotti create/update; giacenze dopo carico/scarico VF
Non in scope	Vendite e clienti TikTok in UI (a differenza di Shopify)
Permessi UI	Collegamento: owner/admin (<code>canManageTikTokConnection</code>)

Callback OAuth: query `?tiktok=connected|error|disconnected` su ritorno frontend Impostazioni.

Variabili aggiuntive in `api/.env.example`: `TIKTOK_APP_URL`, `TIKTOK_OAUTH_CALLBACK_URL`, URL API/auth opzionali.

10. Supabase: database, Auth, Storage, RLS

Prisma

- Schema: `api/prisma/schema.prisma`
- Migration: `npx prisma migrate dev` (locale), deploy in CI/CD API

RLS obbligatoria

Ogni tabella business deve avere RLS attiva. CI esegue `scripts/check-rls.mjs` (workflow `.github/workflows/security.yml`) con anon key: **zero righe** leggibili.

Storage

- Bucket `product-media` public per immagini prodotto
- Bucket `user-avatars` public per foto profilo utente
- Upload via API (service role); avatar: `UserAvatarService`, prodotti: pipeline esistente

Auth

- Utenti creati al provisioning (`admin-tenants.service`)
 - MFA: Supabase TOTP, UI in Impostazioni
-

11. Dominio dati principale

Entità	Note
Tenant	Azienda cliente
User	Profilo app, tenantId , ruolo
Store / Location	Store commerciale; location per stock
Product / ProductVariant	Opzioni generiche; SKU univoco; catalogOrigin , shopifyCatalogLinkKind , shopifyCategoryMetafields , taxonomy categoria
InventoryLevel	variantId × locationId , stati quantità
StockMovement	Audit trail obbligatorio; origine vestiflow_pos per vendite/storni al banco (profilo gestionale)
SupplierOrder	Solo VF
SalesOrder / Customer	Import Shopify, read-only UI; assenti in UI profilo Solo gestionale
ShopifyConnection	Token, scope, stato sync per tenant

Denaro: **interi minor units** (`Money.amountMinor`), mai float.

12. API e permessi tenant

Base URL: `/api/v1`. Header `Authorization` obbligatorio (salvo health).

Rate limiting API VestiFlow (NestJS)

Separato dai limiti Shopify: protezione anti brute-force / DoS sull'**API propria**.

Parametro	Valore	Dove
Limite globale	300 richieste / minuto / IP	<code>ThrottlerModule</code> in <code>api/src/app.module.ts</code>
Guard	<code>ThrottlerGuard</code> su tutte le route	Eccetto route marcate <code>@Public()</code>
Webhook Shopify inbound	Esclusi dal throttling globale	<code>shopify-webhooks.controller.ts</code> — non perdere eventi legittimi

Se un client supera 300 req/min riceve **429**; il frontend lo mappa in `AppError` kind `rate_limited` («Troppe richieste, riprova tra poco»).

Pattern service NestJS:

- Validazione DTO (`class-validator`)
- Filtro `tenantId` da JWT
- Errori → messaggi utente in italiano dove applicabile

Frontend: `HttpClient` + interceptor auth/error; mock disabilitati in prod.

13. Import ed export CSV

Catalogo prodotti

Export	<code>GET /products/export/csv</code> — formato Shopify CSV
Import	<code>POST /products/import/csv</code> — preview + commit; imposta <code>catalogOrigin=vestiflow</code> , <code>linkKind=pushed</code>
UI	Prodotti → Esporta / Importa CSV
Permessi	manager+

Giacenze

Export	<code>GET /inventory/levels/export/csv</code>
Import	<code>POST /inventory/levels/import/csv</code> — rettifiche
Colonne import	SKU, Location (nome esatto), Disponibile
UI	Magazzino → Giacenze
Permessi	manager+

Vendite e clienti

Export CSV dalle liste (filtri rispettati). Sync manuale Shopify: owner/admin.

Ordini fornitori

Metodo	Path	Azione	Permessi
GET	/supplier-orders	Lista paginata (ricerca, filtro stato)	autenticato
GET	/supplier-orders/:id	Dettaglio	autenticato
POST	/supplier-orders	Crea ordine (bozza o inviato)	manager+
PATCH	/supplier-orders/:id	Aggiorna bozza (righe sostituite integralmente)	manager+
POST	/supplier-orders/:id/send	Bozza → inviato	manager+
POST	/supplier-orders/:id/cancel	Annulla (solo bozza o inviato, non ancora ricevuto)	manager+
DELETE	/supplier-orders/:id	Elimina ordine annullato (righe in cascade)	manager+
POST	/supplier-orders/:id/receive	Ricezione merce + movimenti load + push inventario	operativi

Service: `SupplierOrdersService` (`api/src/supplier-orders/`). Ricezione in transazione atomica con `StockMovement` .

UI tenant: form ordine con `select-menu` searchable (fornitore, variante), `date-input` per data attesa, subtotal riga calcolato; dettaglio con **Elimina ordine** se `status=cancelled` . Lista prodotti: stampa etichette multi-select (`ProductLabelPrintService.triggerDirectPrintMany`).

Vendita al banco (profilo Solo gestionale)

Endpoint dedicato per **doppia scansione** al banco: decremento/incremento stock senza creare `SalesOrder` . Disponibile solo se `Tenant.channelProfile === gestionale` (`assertTenantChannelProfile`).

Metodo	Path	Body / azione	Permessi
POST	/inventory/retail-scans	<code>{ code, locationId, action: 'sale' 'return' }</code> — qty fissa 1 per scansione	operativi+

Backend: `InventoryService.registerRetailScan()` (`api/src/inventory/inventory.service.ts`).

- `action: sale` → `StockMovement` tipo `sale` , origine `vestiflow_pos`
- `action: return` → `StockMovement` tipo `return` , origine `vestiflow_pos`
- Lookup variante: stessa semantica di `GET /products/variants/by-code/:code` (SKU o barcode)
- Vendita rifiutata se `available < 1` sulla location
- Migration enum: `0019_vestiflow_pos_movement_origin` → `MovementOrigin.vestiflow_pos`

I tipi `sale` e `return` **non** sono selezionabili nel form manuale **Registra movimento** (solo via retail-scans).

Frontend:

Rotta	Componente	Guard / note
<code>/app/sales/register</code>	<code>RetailSaleRegisterComponent</code>	<code>gestionaleRetailGuard</code> — solo profilo gestionale
<code>/app/sales</code>	lista ordini Shopify	<code>salesHistoryGuard</code> — redirect gestionale → register

- Service HTTP: `InventoryService.registerRetailScan()`
(`src/app/features/inventory/services/inventory.service.ts`)
- Shell: voce **Registra vendita** al posto di **Vendite** se `showGestionaleRetailSales()` (`tenant-channel-profile.model.ts`)
- Label origine movimento: **Vendita negozio** (`inventory-labels.util.ts` → `MovementOrigin.VestiflowPos`)

14. Scanner barcode e componenti UI condivisi

Scanner barcode

- Componente: `shared/components/barcode-scanner` (BarcodeDetector API)
- Flag: `VESTIFLOW_ENABLE_BARCODE_SCANNER`
- Schermate: Cerca giacenza, Giacenze, Registra movimento, **Registra vendita** (vendita + storno), Inventario fisico, Prodotti
- Lookup API: `GET /products/variants/by-code/:code` (SKU o barcode esatto)
- **Pistola USB (keyboard wedge)**: nessuna integrazione dedicata — input HTML + Invio; usata su Registra vendita e altre schermate con campo codice
- Fallback iOS: input manuale

Date e select (form / filtri)

- `shared/components/date-input` — calendario custom al posto di `<input type="date">`; usato in **Report** (filtri periodo) e **Ordini fornitori** (data attesa)
- `shared/components/select-menu` — prop opzionale `searchable` con filtro su label e detail (`shared/utils/select-menu-filter.util`); usata su select **fornitore** e **variante** nel form ordine fornitore; opzioni variante a due righe (titolo + SKU via `variant-select-menu.util`)

15. Generazione guide (HTML/PDF)

```
npm run docs:guide:all
```

Genera:

Output	Contenuto	Audience
public/guide/content.html	Solo guida utente	Tutti (/app/guide)
public/guide/vestiflow-guida.pdf	PDF utente	Download pubblico in-app
src/assets/guide-admin/content-tecnica.html	Solo guida operatore/tecnica	Platform admin
src/assets/guide-admin/vestiflow-guida-tecnica.pdf	PDF tecnico	Download admin
docs/GUIDA-*.html/pdf	Sorgenti in repo	Documentazione offline

Markdown sorgenti:

- docs/GUIDA-UTENTE-VESTIFLOW.md
- docs/GUIDA-OPERATORE-VESTIFLOW.md

Dopo modifica guide: **rigenerare** e committare HTML/PDF prima del deploy.

16. Build, test e CI

Frontend

```
npm run lint
npm run build      # pre-push hook
ng test            # Vitest
```

Backend

```
cd api && npm run lint && npm run build
cd api && npm run test
```

Copertura automatica — Shopify shop change / location / delete

Area	File test
Purge / preview shop change	<code>api/src/shopify/shopify-shop-change.service.spec.ts</code>
Sync location + cleanup onboarding/stale	<code>api/src/shopify/shopify-location-sync.service.spec.ts</code>
Delete prodotto write-through Shopify	<code>api/src/products/products.service.spec.ts</code>
Guard <code>catalogOrigin</code> (update/delete/sync/media)	<code>api/src/products/catalog-origin.util.spec.ts</code>
Wizard UI (anteprima, conferma, disconnect)	<code>src/app/features/integrations/shopify/components/shopify-shop-change-wizard/*.spec.ts</code>
HTTP client shop change / sync location	<code>src/app/features/integrations/shopify/services/shopify-connection.service.spec.ts</code>
E2E wizard (anteprima, step conferma, annulla)	<code>e2e/shopify.spec.ts</code>
Retail scan API (sale/return, profilo, stock)	<code>api/src/inventory/inventory.service.spec.ts</code> , <code>inventory.controller.spec.ts</code>
Guard vendite gestionale vs Shopify	<code>src/app/features/sales/guards/retail-sales.guard.spec.ts</code>
Pagina Registra vendita	<code>src/app/features/sales/pages/retail-sale-register/retail-sale-register.component.spec.ts</code>
HTTP client retail-scans	<code>src/app/features/inventory/services/inventory.service.spec.ts</code>
Profilo canale / label origine movimento	<code>tenant-channel-profile.model.spec.ts</code> , <code>inventory-labels.util.spec.ts</code>

CI GitHub Actions

- `security.yml` : check RLS Supabase (set secrets `SUPABASE_URL` , `SUPABASE_ANON_KEY`)

Estendere pipeline con lint + test + build su PR (best practice repo rules).

17. Deploy produzione

Frontend

1. Imposta env build (`VESTIFLOW_API_BASE_URL` , Supabase anon, flags)
2. `npm run build`
3. Deploy `dist/vestiflow` su Firebase App Hosting
4. `CORS_ORIGINS` API deve includere dominio frontend

Backend

1. Railway (o altro): env da `api/.env.example`
2. `prisma migrate deploy` in release
3. Verifica `PLATFORM_ADMIN_EMAILS` con la tua email operatore
4. `SHOPIFY_APP_URL` = URL API pubblico HTTPS

Post-deploy smoke test

- ☐ Login tenant test
- ☐ Platform admin → Clienti → Nuovo cliente (staging)
- ☐ Profilo canale Shopify e TikTok su tenant test
- ☐ OAuth Shopify su tenant test
- ☐ OAuth TikTok Shop su tenant test (se abilitato)
- ☐ Upload foto profilo + avatar topbar
- ☐ Import catalogo + webhook
- ☐ Tenant **Solo gestionale**: POST retail-scans vendita + storno su variante test
- ☐ Upload immagine prodotto
- ☐ Guida utente + guida tecnica admin

18. Sicurezza e checklist pre-release

- ☐ Nessun secret in git (`.env` gitignored)
 - ☐ RLS attiva su tutte le tabelle (`npm run check:rls` se script root)
 - ☐ `PLATFORM_ADMIN_EMAILS` limitato a operatori fidati
 - ☐ CORS ristretto ai domini produzione
 - ☐ Token Shopify cifrati; revoca su disconnessione
 - ☐ MFA disponibile per admin negozio
 - ☐ Dipendenze: `npm audit` senza high/critical
 - ☐ Guide rigenerate se modificate
-

19. Troubleshooting tecnico

Problema	Azione
<code>isPlatformAdmin</code> false in UI	Verifica email in <code>PLATFORM_ADMIN_EMAILS</code> , ri-login
403 su <code>/admin/tenants</code>	Stesso controllo email lato API
CORS error	Aggiungi origin frontend a <code>CORS_ORIGINS</code>
JWT invalid	Allinea <code>SUPABASE_JWT_SECRET</code> con dashboard Supabase
Webhook non arrivano	URL tunnel/prod raggiungibile; HTTPS; webhook registrati
Import catalogo 429 / throttling Shopify	Attendi 1–2 min; non parallelizzare import; vedi \$9 limiti Shopify; controlla env <code>SHOPIFY_API_*</code>
API VestiFlow 429 (troppi click)	Limite 300 req/min/IP; chiedi al tenant di non ripetere azioni in loop
Immagini prodotto 404	Bucket <code>product-media</code> esiste ed è public
Avatar 404 / upload fallito	Bucket <code>user-avatars</code> esiste ed è public; env <code>SUPABASE_USER_AVATARS_BUCKET</code>
TikTok OAuth fallisce	Verifica <code>TIKTOK_*</code> env, callback URL pubblico HTTPS, app Partner Center attiva
Anon key legge dati	Critico — RLS mancante, fix migration immediato

20. Limitazioni note e roadmap

Area	Stato
Multi-store commerciali in un tenant	Non supportato — un shop = un tenant
Invito utenti / cambio ruolo self-service	Non in UI — solo provisioning iniziale + richiesta operatore
Sync vendite/clienti TikTok Shop	Non implementata — integrazione TikTok ancora parziale
Integrazione TikTok Shop (parità Shopify)	In sviluppo — oggi solo OAuth + push catalogo/giacenze
Bozze ordine Shopify (draft orders)	Non in scope — solo ordini confermati in Vendite
Location manuale senza Shopify	Parziale (location onboarding); sync Shopify consigliato
Cassa / corrispettivi IT nativi	Non previsti — integrazione esterna; VF registra solo stock (gestionale: retail-scans)
Vendita al banco profilo gestionale	Implementata — <code>POST /inventory/retail-scans</code> , UI <code>/app/sales/register</code>
Report server-side avanzati	In evoluzione
Coda bulk Shopify persistente (multi-tenant)	Non implementata — operazioni massicce sincrone HTTP
Notifiche email custom reset password	Config Supabase

Contatti

Manutenzione prodotto: **proprietario VestiFlow** (questo documento è il riferimento operativo interno).